



Московский государственный университет
имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики
Кафедра оптимального управления

Семестровое задание

**«Численное решение краевых задач для систем ОДУ
методом продолжения по параметру»**

студента группы 313

Дроздова Александра Юрьевича

Преподаватель:

Аввакумов С. Н.

Москва, 2026

Содержание

1 Введение	3
2 Постановка задачи	3
3 Метод решения	4
3.1 Сведение краевой задачи к нелинейному уравнению	4
3.2 Гомотопия	5
3.3 Уравнение в вариациях	5
3.4 Алгоритм	6
4 Реализация	6
4.1 Стек технологий	6
4.2 Структура проекта	7
4.3 Главное окно «Решение»	7
4.4 Карточка графика	8
4.5 Таблица значений	8
4.6 Вкладка «Параметры»	9
4.7 Темы и локализация	9
4.8 Многопоточность	9
4.9 Валидация ввода	10
5 Полученные результаты	10
5.1 Тестирование на учебных задачах	10
5.2 Тестирование на задачах из статьи	10
5.3 Автоматизированные тесты	11
5.4 Итоги	11
Список литературы	12

1 Введение

Краевые задачи для систем обыкновенных дифференциальных уравнений возникают во многих прикладных областях: задачи небесной механики (двух тел, многих тел), теория предельных циклов в нелинейной динамике, оптимальное управление с краевыми условиями на состояние и управление. В отличие от задачи Коши, где известно начальное состояние системы, в краевой задаче условия на решение наложены на двух (или более) различных моментах времени, что делает её существенно сложнее.

Один из классических подходов к численному решению таких задач — метод стрельбы. Идея метода состоит в том, чтобы свести краевую задачу к задаче поиска нулей нелинейного отображения относительно недостающего начального условия. Однако прямое применение метода Ньютона часто терпит неудачу: метод требует «хорошего» стартового приближения, которое в практических задачах неизвестно.

Решение этой проблемы — метод продолжения по параметру (гомотопия). Он строит непрерывный путь от заданного стартового приближения до искомого решения, превращая поиск нулей нелинейного уравнения в задачу Коши, которую можно решить устойчивыми численными методами.

В работе [1] С.Н. Аввакумов и Ю.Н. Киселев изложили несколько алгоритмов оптимального управления, основанных на этом подходе, и привели три модельных примера: задачу двух тел, исследование предельных циклов Эквейлера и трёхкратный интегратор. Эти примеры стали тестовой базой настоящей работы.

2 Постановка задачи

Цель семестрового задания — разработать программу с графическим интерфейсом, реализующую численное решение краевых задач для систем ОДУ методом продолжения по параметру.

Дана система обыкновенных дифференциальных уравнений

$$\dot{x} = f(t, x), \quad t \in [a, b], \quad x \in \mathbb{R}^n,$$

с краевыми условиями

$$R(x(a), x(b)) = 0, \quad R : \mathbb{R}^{2n} \rightarrow \mathbb{R}^n.$$

Требуется найти функцию $x(t)$, удовлетворяющую как уравнению, так и условиям на концах отрезка.

Программа должна обеспечивать:

- ввод произвольной системы ОДУ и краевых условий пользователем в символьной форме;
- численное решение методом продолжения по параметру с возможностью выбора метода интегрирования и точности;
- визуализацию решения в виде графика по времени и в виде фазовой плоскости;
- вывод таблицы значений с настраиваемым шагом;
- встроенную библиотеку модельных задач (учебные примеры и примеры из статьи [1]);
- сохранение и загрузку задач в формате JSON;
- локализованный интерфейс (русский и английский языки), переключение светлой и тёмной темы;
- корректную обработку ошибок ввода с понятными сообщениями.

3 Метод решения

3.1 Сведение краевой задачи к нелинейному уравнению

Введём искомый параметр $p = x(t^*) \in \mathbb{R}^n$, где $t^* \in [a, b]$ — произвольная точка (как правило, $t^* = a$). Если p известно, то значения $x(a; p)$ и $x(b; p)$ определяются интегрированием задачи Коши $\dot{x} = f(t, x)$, $x(t^*) = p$ от t^* до концов отрезка.

Определим функционал невязки

$$\Phi(p) := R(x(a; p), x(b; p)).$$

Тогда исходная краевая задача эквивалентна нелинейному уравнению

$$\Phi(p) = 0. \tag{1}$$

Уравнение (1) соответствует формуле (1.14) статьи [1].

3.2 Гомотопия

Прямое решение уравнения (1) методом Ньютона неустойчиво. Метод продолжения по параметру строит однопараметрическое семейство задач $\{\Phi(p) = (1 - \mu)\Phi(p_0)\}_{\mu \in [0,1]}$, такое что:

- при $\mu = 0$ известно решение $p(0) = p_0$ (стартовое приближение);
- при $\mu = 1$ имеем искомое $\Phi(p(1)) = 0$.

Дифференцируя тождество $\Phi(p(\mu)) = (1 - \mu)\Phi(p_0)$ по μ , получаем уравнение в дифференциальной форме:

$$\frac{dp}{d\mu} = -[\Phi'(p)]^{-1}\Phi(p_0), \quad p(0) = p_0, \quad \mu \in [0, 1]. \quad (2)$$

Уравнение (2) соответствует формуле (1.15) статьи [1] и в её терминологии называется *внешней задачей*.

Это задача Коши относительно $p(\mu)$, и она решается стандартными методами численного интегрирования. После интегрирования от $\mu = 0$ до $\mu = 1$ получаем приближение $p_1 = p(1)$. Если $\|\Phi(p_1)\| > \varepsilon$, итерация повторяется с новым стартом p_1 и зафиксированной невязкой $\Phi(p_1)$.

3.3 Уравнение в вариациях

Вычисление матрицы Якоби $\Phi'(p)$ выполняется по цепному правилу:

$$\Phi'(p) = R_{x_a}(x(a), x(b)) \cdot X(a) + R_{x_b}(x(a), x(b)) \cdot X(b),$$

где $X(t) := \partial x(t)/\partial p$ — матрица вариаций размера $n \times n$. Дифференцируя уравнение $\dot{x} = f(t, x)$ по p , получаем уравнение в вариациях:

$$\dot{X} = f_x(t, x) \cdot X, \quad X(t^*) = I_n. \quad (3)$$

Уравнения (3) интегрируются совместно с основным уравнением. Расширенная система имеет размерность $n + n^2$:

$$\begin{cases} \dot{x} = f(t, x), \\ \dot{X} = f_x(t, x) \cdot X, \end{cases} \quad x(t^*) = p, \quad X(t^*) = I_n.$$

Эта система соответствует формуле (1.16) статьи [1] и в её терминологии называется *внутренней задачей*.

После одного интегрирования от t^* до a и от t^* до b получаются сразу все необходимые величины: $x(a)$, $x(b)$, $X(a)$, $X(b)$.

3.4 Алгоритм

Один внешний шаг метода:

1. На текущем p вычислить $\Phi(p)$ интегрированием расширенной системы.
2. Если $\|\Phi(p)\| < \varepsilon$ — завершить, p — решение.
3. Зафиксировать $\Phi_0 = \Phi(p)$.
4. Решить ОДУ (2) на отрезке $\mu \in [0, 1]$. На каждом шаге внутри численного интегратора:
 - (a) проинтегрировать расширенную систему, получить $x(a)$, $x(b)$, $X(a)$, $X(b)$;
 - (b) вычислить $\Phi'(p) = R_{x_a}X(a) + R_{x_b}X(b)$;
 - (c) решить линейную систему $\Phi'(p) \cdot v = -\Phi_0$ относительно $v = dp/d\mu$.
5. Положить $p \leftarrow p(\mu = 1)$.
6. Перейти к шагу 1 (но не более `max_iter` раз).

После сходимости выполняется ещё одно финальное интегрирование уже только основного уравнения $\dot{x} = f(t, x)$ с режимом плотного вывода (`dense_output`), что позволяет получать значения $x(t)$ в произвольных точках для построения таблиц с настраиваемым шагом.

4 Реализация

4.1 Стек технологий

Программа реализована на языке Python (3.10+) с использованием следующих библиотек:

- **NumPy** (2.4) — работа с многомерными массивами, линейная алгебра;
- **SciPy** (1.17) — численное интегрирование (`scipy.integrate.solve_ivp`);
- **SymPy** (1.14) — символьное вычисление якобианов и компиляция выражений в численные функции (`lambdify`);
- **Matplotlib** (3.10) — построение графиков;
- **PyQt6** (6.11) — графический интерфейс.

Использование SymPy для якобианов позволяет вычислять матрицы f_x , R_{x_a} , R_{x_b} точно (без ошибок численного дифференцирования), а `lambdify` компилирует символьные выражения в быстрые NumPy-функции. Парсинг пользовательских формул выполняется через `sympy.parsing.sympy_parser.parse_expr` (без использования `eval`, что обеспечивает безопасность).

4.2 Структура проекта

Проект разделён на три логических слоя.

Ядро (`solver.py`) — реализация метода стрельбы с гомотопией. Не зависит от GUI и может использоваться как библиотека.

Данные (`examples/library.py`) — словарь из девяти модельных задач: три учебных (линейная, экспонента, гармонический осциллятор) и шесть из статьи [1].

Графический интерфейс (`gui/app.py`, `gui/i18n.py`) — основное окно, четыре вкладки («Решение», «Параметры», «Автор», «Помощь»), темы и локализация.

4.3 Главное окно «Решение»

Главная вкладка разделена сплиттером на две колонки. В левой колонке расположены поля ввода:

- размерность системы n (от 1 до 10), при изменении которой поля для уравнений, краевых условий и начального приближения создаются и удаляются динамически;
- уравнения $\dot{x}_i = f_i(t, x)$ в символьной форме (переменные $x_1, \dots, x_n$, время t);

- краевые условия $R_i = 0$ с обозначениями **xa0**, **xa1**, ... для значений $x_i(a)$ и **xb0**, **xb1**, ... для $x_i(b)$;
- границы отрезка $[a, b]$ и точка старта t^* ;
- начальное приближение $p_0 = x(t^*)$;
- тулбар с кнопками вызова библиотеки примеров, открытия и сохранения задачи в формате JSON;
- кнопка «Решить» и поле статуса.

В правой колонке отображается результат: график решения (matplotlib, встроенный через FigureCanvasQTAgg) и таблица значений.

4.4 Карточка графика

Над графиком расположен ряд функциональных кнопок:

- **«Фаза»** — переключение в режим фазовой плоскости. При $n \geq 2$ появляются два выпадающих списка для выбора переменных, отображаемых по осям X и Y . Вместо нескольких кривых $x_i(t)$ строится одна траектория $x_j(x_i)$.
- **«Точки»** — отображение узлов сетки численного интегратора.
- **«Цвет»** — индивидуальный выбор цвета для каждой переменной через стандартный диалог цвета.
- **«Оси»** — ручное задание границ осей. При активации появляется строка с полями **xmin**, **xmax**, **ymin**, **ymax**; пустые поля означают автоматический масштаб.

4.5 Таблица значений

Над таблицей расположено поле «Шаг Δt ». Если оно пустое, таблица строится прорежением узлов сетки интегратора (приблизительно 50 строк). Если задано числовое значение шага, таблица строится на равномерной сетке от a до b с использованием интерполятора, возвращённого солвером (режим **dense_output**). Шаг должен быть не больше длины отрезка, иначе он игнорируется. Строки нумеруются автоматически.

4.6 Вкладка «Параметры»

Вкладка содержит две карточки — для внутренней (интегрирование расширенной системы) и внешней (метод продолжения) задач. В каждой пользователь может выбрать:

- метод интегрирования: RK45 (Рунге-Кутта 4(5), универсальный), Radau (неявный, для жёстких систем), LSODA (автоматический выбор);
- точность интегрирования (используется как `rtol` и `atol`), значения от 10^{-2} до 10^{-12} .

Внешняя задача дополнительно содержит максимальное число итераций.

По умолчанию используются точности 10^{-6} для внутренней задачи и 10^{-4} для внешней — те же значения, что приведены в статье [1] для всех трёх примеров. Метод RK45 в `scipy.integrate.solve_ivp` (явный метод Рунге-Кутта 4(5) Дорманда-Принса) является современным аналогом метода Рунге-Кутта-Фельберга `rkf-45`, использованного авторами статьи в среде Maple.

Эмпирическое правило: точность внутренней задачи должна быть жёстче точности внешней, чтобы внешний метод «видел» реальную ошибку.

4.7 Темы и локализация

Реализованы две темы (светлая и тёмная), переключаемые кнопкой / в шапке. Все цвета хранятся в двух словарях с одинаковыми ключами, а Qt-стилевая таблица собирается f-строкой. При смене темы пересчитывается стиль всех виджетов и перерисовывается график (matplotlib не управляется через Qt CSS).

Локализация (русский и английский языки) реализована через словарь `STRINGS` в `gui/i18n.py`. Все строки — от заголовков до длинных HTML-блоков справки — хранятся в одном файле. Переключение языка кнопкой EN/RU обновляет тексты без пересоздания виджетов; ширины ключевых элементов фиксированы, чтобы интерфейс не «прыгал».

4.8 Многопоточность

Решение краевой задачи на сложных примерах (например, трёхкратный интегратор) занимает несколько секунд. Чтобы интерфейс не замораживался, вычисление выполняется в отдельном потоке (`threading.Thread`), а результат передаётся в основной поток через

сигнал PyQt (`pyqtSignal`). Сигналы Qt автоматически обеспечивают thread-safe передачу данных между потоками.

4.9 Валидация ввода

Перед запуском вычисления все поля проверяются: уравнения и краевые условия не должны быть пустыми, p_0 — числа, выполнено $a < b$, точка t^* лежит в $[a, b]$, максимум итераций — целое число. При обнаружении ошибки появляется диалоговое окно с конкретным указанием некорректного поля.

Дополнительно солвер сообщает об ошибках вычисления: вырожденность матрицы Якоби, расходимость метода, неудача внутреннего интегрирования — всё это превращается в понятное сообщение, отображаемое в строке статуса.

5 Полученные результаты

5.1 Тестирование на учебных задачах

Программа протестирована на трёх учебных задачах, имеющих точное аналитическое решение:

- $x'' = 0$, $x(0) = 0$, $x(1) = 1$ — ожидаемое решение $x(t) = t$;
- $x' = x$, $x(0) = 1$ — ожидаемое решение $x(t) = e^t$;
- $x'' + x = 0$, $x(0) = 0$, $x(\pi) = 0$ — ожидаемое решение $x(t) = \sin t$.

На всех трёх задачах численное решение совпадает с аналитическим в пределах заданной точности.

5.2 Тестирование на задачах из статьи

Программа верно воспроизводит результаты, приведённые в статье [1]:

- **Задача двух тел** (пример 1) — найдены оба решения, соответствующие двум различным траекториям, возвращающим тело в окрестность исходной точки;

- **Предельные циклы Эквейлера** (пример 2) — найдены три предельных цикла различной амплитуды для гладкой системы со специальной нелинейностью;
- **Трёхкратный интегратор** (пример 3) — задача оптимального управления со сглаженной функцией насыщения и нестандартной точкой старта $t^* = b$, что потребовало двунаправленного интегрирования.

5.3 Автоматизированные тесты

В директории `tests/` реализованы три автоматических теста (`test_linear`, `test_exponential`, `test_two_body`), сравнивающие результат программы с известным аналитическим (или табличным) решением. Все тесты проходят.

5.4 Итоги

Получена работающая программа с графическим интерфейсом, способная решать произвольные краевые задачи для систем ОДУ методом стрельбы с гомотопией. Программа поддерживает девять встроенных модельных задач, сохранение и загрузку задач в формате JSON, две темы оформления и два языка интерфейса. Реализована корректная обработка ошибок ввода и сбоя вычисления. Исходный код размещён в публичном репозитории на GitHub.

Список литературы

- [1] Аввакумов С. Н., Киселев Ю. Н. Некоторые алгоритмы оптимального управления // Труды Института математики и механики УрО РАН. 2006. Т. 12, № 2. С. 3–17.
- [2] Киселев Ю. Н., Аввакумов С. Н., Орлов М. В. Оптимальное управление. Линейная теория и приложения. М.: Издательский отдел факультета ВМиК МГУ им. М. В. Ломоносова, 2007. 270 с.
- [3] Virtanen P. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. 2020. Vol. 17. P. 261–272.
- [4] Meurer A. et al. SymPy: Symbolic Computing in Python // PeerJ Computer Science. 2017. Vol. 3. e103.
- [5] Дроздов А. Ю. Исходный код проекта. https://github.com/nknxwn/prac_spring26.